FILE: C:\HCL_TRAINING\server\src\controllers\pipelineController.js

LINES: 255 | SIZE: 7.6 KB | LANGUAGE: JS

MODULE: 7-Stage Kanban Board Transitions, Bulk Moves, and Undo API

```
const Application = require('../models/Application');
const ScreeningResult = require('../models/ScreeningResult');
const Notification = require('../models/Notification');
const { logActivity } = require('../services/auditLogger');

const PIPELINE_STAGES = ['New', 'Screening', 'Shortlisted', 'Interview', 'Offer', 'Hired
> ', 'Rejected'];

// @desc    Get Kanban board applications organized by stage
// @route   GET /api/pipeline
const getPipelineBoard = async (req, res, next) => {
  try {
    const { jobId, recruiterId, search } = req.query;
    const filter = { organization: req.organizationId };

    if (jobId && jobId !== 'all') {
      filter.jobId = jobId;
    }
    if (recruiterId && recruiterId !== 'all') {
      filter.assignedRecruiter = recruiterId;
    }

    const applications = await Application.find(filter)
      .populate('candidateId', 'name email location totalExperienceYears skills')
      .populate('jobId', 'title department')
      .populate('assignedRecruiter', 'name email avatar')
      .sort({ updatedAt: -1 });

    // Fetch screening scores for these applications
    const candIds = applications.map((a) => a.candidateId ? a.candidateId._id : null).fi
> lter(Boolean);
    const screeningDocs = await ScreeningResult.find({
      candidateId: { $in: candIds },
      organization: req.organizationId,
      ...(jobId && jobId !== 'all' ? { jobId } : {}),
    }).select('candidateId jobId overallScore componentScores matchedRequiredSkills miss
> ingRequiredSkills');

    const scoreMap = {};
    screeningDocs.forEach((doc) => {
      scoreMap[`${doc.candidateId}_${doc.jobId}`] = doc;
    });

    // Group by stages
    const columns = {};
    PIPELINE_STAGES.forEach((stage) => {
      columns[stage] = {
        stage,
        count: 0,
        candidates: [],
      };
    });

    applications.forEach((app) => {
      if (!app.candidateId) return;

      // Filter by search query if present
      if (search) {
        const queryLower = search.toLowerCase();
        const candName = (app.candidateId.name || '').toLowerCase();
        const candEmail = (app.candidateId.email || '').toLowerCase();
        if (!candName.includes(queryLower) && !candEmail.includes(queryLower)) {
          return;
        }
      }

```

```
64 |       const stageKey = columns[app.stage] ? app.stage : 'New';
65 |       const screening = scoreMap[`${app.candidateId._id}_${app.jobId._id}`];
66 |
67 |       columns[stageKey].candidates.push({
68 |         _id: app._id,
69 |         candidate: app.candidateId,
70 |         job: app.jobId,
71 |         assignedRecruiter: app.assignedRecruiter,
72 |         stage: app.stage,
73 |         stageHistory: app.stageHistory,
74 |         notesCount: (app.notes || []).length,
75 |         score: screening ? screening.overallScore : null,
76 |         screeningId: screening ? screening._id : null,
77 |         updatedAt: app.updatedAt,
78 |       });
79 |
80 |       columns[stageKey].count += 1;
81 |     });
82 |
83 |     res.status(200).json({
84 |       success: true,
85 |       data: {
86 |         stages: PIPELINE_STAGES,
87 |         columns,
88 |         totalCandidates: applications.length,
89 |       },
90 |     });
91 |   } catch (error) {
92 |     next(error);
93 |   }
94 | };
95 |
96 | // @desc    Update single application stage (Drag-and-drop or manual move)
97 | // @route   PATCH /api/pipeline/:applicationId/stage
98 | const updateCandidateStage = async (req, res, next) => {
99 |   try {
100|     const { stage, note } = req.body;
101|     const { applicationId } = req.params;
102|
103|     if (!PIPELINE_STAGES.includes(stage)) {
104|       return res.status(400).json({ success: false, message: `Invalid stage. Allowed: ${
    > PIPELINE_STAGES.join(', ')}` });
105|     }
106|
107|     const app = await Application.findOne({
108|       _id: applicationId,
109|       organization: req.organizationId,
110|     }).populate('candidateId', 'name').populate('jobId', 'title');
111|
112|     if (!app) {
113|       return res.status(404).json({ success: false, message: 'Application record not fou
    > nd' });
114|     }
115|
116|     const previousStage = app.stage;
117|     app.stage = stage;
118|     app.stageHistory.push({
119|       stage,
120|       previousStage,
121|       changedBy: req.user._id,
122|       changedAt: new Date(),
123|       note: note || `Stage updated from ${previousStage} to ${stage}`,
124|     });
125|
126|     await app.save();
127|
128|     // Notify assigned recruiter if not the one changing
129|     if (app.assignedRecruiter && app.assignedRecruiter.toString() !== req.user._id.toStr
    > ing()) {
130|       await Notification.create({
131|         user: app.assignedRecruiter,
132|         title: 'Candidate Pipeline Stage Changed',
```

```
133 |          message: `${app.candidateId.name} was moved to ${stage} for ${app.jobId.title}`,
134 |          type: 'stage_change',
135 |          link: `/candidates/${app.candidateId._id}`,
136 |          organization: req.organizationId,
137 |        });
138 |      }
139 |
140 |      await logActivity({
141 |        organizationId: req.organizationId,
142 |        userId: req.user._id,
143 |        action: 'STAGE_CHANGED',
144 |        entityType: 'Application',
145 |        entityId: app._id,
146 |        details: { candidate: app.candidateId.name, previousStage, newStage: stage },
147 |        req,
148 |      });
149 |
150 |      res.status(200).json({
151 |        success: true,
152 |        data: app,
153 |      });
154 |    } catch (error) {
155 |      next(error);
156 |    }
157 |  };
158 |
159 |  // @desc    Bulk update stage for multiple candidates
160 |  // @route   POST /api/pipeline/bulk-stage
161 |  const bulkUpdateStage = async (req, res, next) => {
162 |    try {
163 |      const { applicationIds, targetStage, note } = req.body;
164 |
165 |      if (!Array.isArray(applicationIds) || applicationIds.length === 0) {
166 |        return res.status(400).json({ success: false, message: 'Please provide at least on
    >   e candidate application' });
167 |      }
168 |
169 |      if (!PIPELINE_STAGES.includes(targetStage)) {
170 |        return res.status(400).json({ success: false, message: `Invalid stage: ${targetSta
    >   ge}` });
171 |      }
172 |
173 |      const updated = await Promise.all(
174 |        applicationIds.map(async (id) => {
175 |          const app = await Application.findOne({ _id: id, organization: req.organizationI
    >   d });
176 |          if (app) {
177 |            const prev = app.stage;
178 |            app.stage = targetStage;
179 |            app.stageHistory.push({
180 |              stage: targetStage,
181 |              previousStage: prev,
182 |              changedBy: req.user._id,
183 |              changedAt: new Date(),
184 |              note: note || `Bulk stage update to ${targetStage}`,
185 |            });
186 |            return await app.save();
187 |          }
188 |          return null;
189 |        })
190 |      );
191 |
192 |      await logActivity({
193 |        organizationId: req.organizationId,
194 |        userId: req.user._id,
195 |        action: 'BULK_STAGE_CHANGED',
196 |        entityType: 'Application',
197 |        details: { count: applicationIds.length, targetStage },
198 |        req,
199 |      });
200 |
201 |      res.status(200).json({
```

```
202 |       success: true,
203 |       message: `Updated ${applicationIds.length} candidate(s) to ${targetStage}`,
204 |     });
205 |   } catch (error) {
206 |     next(error);
207 |   }
208 | };
209 |
210 | // @desc    Undo stage change
211 | // @route   POST /api/pipeline/:applicationId/undo
212 | const undoStageChange = async (req, res, next) => {
213 |   try {
214 |     const { applicationId } = req.params;
215 |     const app = await Application.findOne({ _id: applicationId, organization: req.organi
    >   zationId });
216 |
217 |     if (!app) {
218 |       return res.status(404).json({ success: false, message: 'Application not found' });
219 |     }
220 |
221 |     if (app.stageHistory.length < 2) {
222 |       return res.status(400).json({ success: false, message: 'No prior stage history to
    >   revert to' });
223 |     }
224 |
225 |     // Pop the latest history and revert to prior
226 |     const lastItem = app.stageHistory[app.stageHistory.length - 1];
227 |     const previous = lastItem.previousStage || 'New';
228 |
229 |     app.stage = previous;
230 |     app.stageHistory.push({
231 |       stage: previous,
232 |       previousStage: lastItem.stage,
233 |       changedBy: req.user._id,
234 |       changedAt: new Date(),
235 |       note: 'Reverted stage change',
236 |     });
237 |
238 |     await app.save();
239 |
240 |     res.status(200).json({
241 |       success: true,
242 |       data: app,
243 |     });
244 |   } catch (error) {
245 |     next(error);
246 |   }
247 | };
248 |
249 | module.exports = {
250 |   getPipelineBoard,
251 |   updateCandidateStage,
252 |   bulkUpdateStage,
253 |   undoStageChange,
254 | };
255 |
```